LPP PREGUNTAS TEMA #03

1. **¿Qué es la concurrencia?**

La concurrencia es la capacidad de un sistema para ejecutar simultáneamente múltiples procesos que comparten recursos: memoria, registros, cpu…

2. **¿Cuáles son las principales características de la programación concurrente?**

Permite dividir un programa en múltiples tareas que pueden ejecutarse de manera independiente, compartir recursos y comunicarse entre sí, lo que facilita una mayor utilización de los recursos del sistema.

3. **¿Qué es el paralelismo?**

El paralelismo consiste en la ejecución simultánea de varias tareas o procesos con el objetivo de mejorar el rendimiento y reducir el tiempo de ejecución.

4. **¿Cuáles son las principales características de la programación paralela?**

Incluye la ejecución simultánea de tareas, el uso de múltiples procesadores, y requiere sincronización y coordinación para evitar conflictos al acceder a recursos compartidos.

5. **¿Cuáles son las principales características de la programación distribuida?**

Implica la ejecución de tareas en múltiples computadoras conectadas por una red, con memoria y recursos separados, y requiere mecanismos para comunicación, sincronización y tolerancia a fallos.

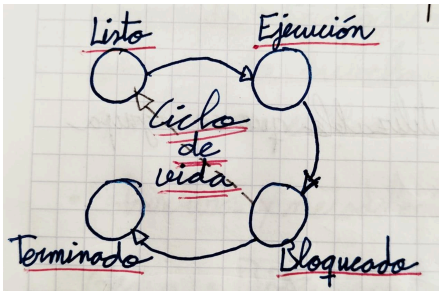
6. **¿Cuáles son las diferencias entre programación concurrente, programación paralela y programación distribuida?**

La concurrente gestiona tareas que se ejecutan de forma intercalada, la paralela ejecuta tareas simultáneamente en varios núcleos, y la distribuida reparte tareas entre distintos sistemas físicos.

7. **¿Qué es un proceso?**

Es una instancia de un programa en ejecución, con su propio espacio de memoria, recursos y contexto de ejecución.

8. **¿Cuáles son los estados del ciclo de vida de un proceso?**



El proceso, una vez es creado, pasa de "**nuevo**" a "**listo**", donde espera a ser ejecutado. Durante la "**ejecución**" puede ser "**bloqueado**" por quedar a la espera de algo (ej: E/S), una vez sale de "**bloqueado**" vuelve a estar "**listo**", donde espera a seguir en "**ejecución**". Una vez se termina de ejecutar, es "**terminado**", donde se liberan los recursos que tenia asociados.

9. **¿Qué es un proceso ligero (Thread)?**

Es una unidad básica de ejecución dentro de un proceso que comparte el mismo espacio de memoria, registros y cpu, pero puede ejecutarse concurrentemente con otros hilos del mismo proceso.

10. **¿Para qué se utilizan las condiciones de Bernstein?**

Se usan para determinar si dos o más instrucciones pueden ejecutarse en paralelo sin interferencias ni conflictos de datos.

11. **¿Cómo se definen las condiciones de Bernstein?**

Se basan en tres conjuntos para cada instrucción: lectura (L), escritura (E) y condiciones que garantizan independencia entre instrucciones si no hay intersecciones entre sus conjuntos L y E.

- $L(Sx) \cap E(Sy) = \emptyset$
- $E(Sx) \cap L(Sy) = \emptyset$
- $E(Sx) \cap E(Sy) = \emptyset$

12. **Utilizando las condiciones de Bernstein construya el grafo de dependencias de las siguientes sentencias:**

S1: c = x * x → W={c}, R={x}
S2: m1 = a * c → W={m1}, R={a,c}
S3: m2 = b * x → W={m2}, R={b,x}
S4: z = m1 + m2 → W={z}, R={m1,m2}
S5: y = z + c → W={y}, R={z,c}

Dependencias:
S1, S3 → S2 → S4 → S5

13. **¿Cómo puede ser el orden de ejecución de un conjunto de instrucciones?**

Solamente puede ser secuencial, ya que las instrucciones solamente se pueden ejecutar una tras otra.

14. **¿Qué significa que los programas concurrentes puedan tener un comportamiento no determinista?**

Significa que el resultado obtenido puede variar en cada ejecución debido al orden impredecible en que los hilos o procesos ejecutan las instrucciones.

15. **¿Cuáles son los dos problemas principales inherentes a la programación paralela?**

Condiciones de carrera y problemas de sincronización, como deadlocks e inanición.

16. **¿Qué es una condición de carrera (race condition)?**

Es una situación en la que dos o más hilos acceden simultáneamente a un recurso compartido, generalmente una dirección de memoria, y el resultado depende del orden en que se ejecutan las instrucciones.

17. **¿Qué es la exclusión mutua (mutual exclusion) *(mutex)*?**

Es un mecanismo que asegura que solo un hilo pueda acceder a una sección crítica del código al mismo tiempo.

18. **¿Qué es una sección crítica (critical section)?**

Es una parte del código que necesita de sincronización para evitar inconsistencias, así como la condición de carrera.

19. **¿En qué consiste la serialización/secuencialización o condición de sincronización?**

Consiste en forzar un orden específico de ejecución entre hilos o procesos para mantener la coherencia y evitar conflictos.

20. **¿En qué consiste un interbloqueo o abrazo mortal (deadlock)?**

Es una situación en la que varios hilos quedan esperando indefinidamente unos por otros para liberar recursos, bloqueando el sistema.

21. **¿En qué consiste la inanición (starvation)?**

Es una condición donde un hilo nunca obtiene acceso a los recursos necesarios porque otros hilos acaparan dichos recursos constantemente.